

A Scratchpad Spiking Neural Network Accelerator

Rasha Karakchi

Dept. Computer Science and Engineering
University of South Carolina
Columbia, SC
karakchi@cec.sc.edu

Abstract—Spiking Neural Networks (SNNs) have recently been used as a computational model for applications such as deep learning, image recognition and machine learning. Similar to the biological brain, SNN neurons depend on the membrane level to fire an output. If the level exceeds a specified threshold, the neuron sends an output to activate next neurons. This leads to an unbalanced workload among the neurons. The dynamically-changing membrane level is stored inside a neuron. In hardware, this storage can be implemented as a register or on-chip memory, which determines the amount of consumed resources and, in turns, affects the network scalability. SNN accelerators have recently been implemented on UltraScale FPGAs devices for high-performance purposes. On-chip memories on these devices are classified as distributed memory, Block RAMs (BRAMs) and Ultra RAMs (URAMs). In this paper, we explored the impact of using different on-chip memories to store the membrane level of SNN neurons. We implemented a parameterizable Spiking Neural networkK (SINK) accelerator where the network capacity and weight_width are parameters. SINK has the ability to run in four different modes based on the memory type. We ran SINK on UltraScale Zync104 FPGA device and measure the utilization of the hardware resources (LUTs), registers, memory, power consumption and performance. The results show that URAM can be the best fit to store the membrane level, since it used 30%, 11% and 2% less LUTs, Regs, and power 2% respectively comparing with BRAM and distributed memory

Index Terms—Spiking Neural Network, FPGA, Onchip Memory, URAM, BRAM, Distributed Memory

I. INTRODUCTION

Spiking Neural Networks (SNNs) have become a hot topic of research because of their similarity to the way human brain functions. This allows them to model various emerging applications such applications related to speech recognition, image classification and automata control [1]. The biological behavior of these networks have recently been attracted for hardware acceleration. Accelerators such as Field-Programmable Gate Arrays (or FPGAs) are domain-specific architectures which are well-known by their capabilities to be programmed for specific purposes. Due to the large amount of parameters needed to be stored and huge number of operations to be computed, executing spiking neural networks on FPGAs have been limited by several factors that negatively impact its scalability [2].

The major difference between conventional neural networks (CNNs) and spiking neural networks (SNNs) is that spiking neural network depends on the membrane level of the neuron to fire a spike [3]. When the membrane potential reaches a specified threshold, the neuron fires, and generates a signal that travels to other neurons, which in turn, increase or decrease their potentials in response to this signal. The membrane level should be stored in a memory and changes dynamically over the time. The amount of the membrane level determines which neurons fire outputs at run time. This behavior leads to spatiotemporal sparsity. At one time, only neurons whose membrane levels exceeds the threshold will produce an output, while others remain offline until the membrane condition becomes true (unlike CNNs where all neurons run simultaneously) [4]. This sparsity causes some sort of unstructured and imbalance workload [5]. In hardware, sparse architectures are considered extremely costly in terms of hardware resources, power consumption and efficiency, specially for the unstructured datasets because of the unpredictable memory accesses which can be affected by the temporal and spatial locality of the memory [6]. For this reason, the way that neurons mapped onto the device (FPGA) and the type of memory type that is utilized to implement the internal temporal memory significantly impacts the overall performance. Memories are divided into three main categories: registers, on-chip memory and off-chip memory [7]. Registers are a type of computer memory built directly into the processor element (PE) that is used to store and manipulate data during the execution of instructions. Registers are fast, however they are limited in numbers. Onchip (Scratchpad) memories such as cache, BRAM (Block RAM), URAM (UltraRAM) and Dist (distributed memory) are data memory residing on-chip that is mapped into an address space disjoint from the off-chip memory but connected to same address and data buses [8]. Onchip memories allow fast access to their residing data comparing to off-chip memories such as HBMs (High Bandwidth Memories) and DDRs (Dynamic RAMs) which relatively require longer access time [9].

In this paper, we investigate the impact of employing registers and on_chip memories as the internal temporal memory of the neurons. We implemented a parameterizable spiking

neural network accelerator using verilog Hardware Description Language (HDL). Our design is adapted from S2N2 Leak Integrate Fire LIF-model published in [10]. Our accelerator is designed as a parameterized accelerator where the network size, weight width and type of the internal memory for membrane level are variables. While network size and weight width are variables, the membrane-memory can be selected as either register, Distributed memory, BRAM or URAM. We measured the hardware resources, power consumption, performance and scalability for each version. This work is comprised of two main contributions:

- 1) a parameterizable RTL-based Spiking Neural networkK accelerator, SINK, which is comprised of an array of hardware LIF-based neurons. SINK has the ability to run in four different modes depending on the selected on_chip memory which are register-based, URAM-based, BRAM-based and Dist-based.
- 2) analysis of the tradeoffs between network capacity, hardware resources, power consumption and on_chip memory as well as a comparison to the state-of-the-art FPGA-based SNN designs.

II. RELATED WORK

Several research group have recently focused on accelerating spiking neural networks on FPGAs [2], [8], [10]–[15]. Some research group [11], [12], [14] designed spiking neural network similar to Network on Chip (NoC) where neurons are grouped into cores where synapses behaved similar to the router. However interconnections and the network capacity bottleneck the possibility to scale up the networks. S2N2 [10] is spiking neural network accelerator that is designed based on FINN framework [16] that supports event-based processing of SNNs. S2N2 has the capability to stream spikes into neurons in a high throughput rate by performing tick-batches approach. SyncNN [8] proposed a novel synchronous event-driven SNN reconfigurable inference engine and evaluated multiple SNN models on multiple FPGAs. Other research groups [5] achieved high acceleration through exploring spatial-temporal parallelism by unrolling the computations in both spatial and time dimensions. However, their approach violates the time-related sequential nature of the membrane level voltage update behavior [17]. Firefly is an RTL-based spiking neural network accelerator that focused on optimizing spatial parallelism on FPGA through optimizing the Multiply and Adder (MAC) unit by incorporating DSP optimization techniques into SNN synaptic operations, as well as using on-chip memory design to store the weight and improve memory efficiency. In this paper, we focus on increasing the SNN scalability though enabling the membrane-level memory access to on-chip memory which improves the memory efficiency.

III. BACKGROUND

Spiking neural networks are the third generation of neural networks whose behavior is similar to the biological human brain [10], [18], [19]. Spiking neural network operates on *spikes* happening at discrete events which is different from

other neural networks which operate at continuous values. As shown in Figure 1, the input of each neuron is represented by *spikes*, which are binary values coming from predecessors (previous neurons). Each active neuron produces spikes which are fired towards its successors (next neurons). Similar to biological brains, neurons are connected via *synapses*. To fire a spike, a neuron depends on the incoming spikes and membrane level which determines the neuron's current state. Unlike other

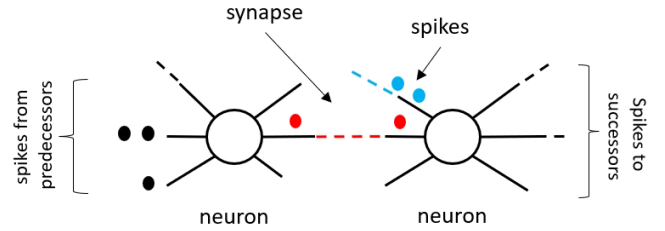


Fig. 1. SNN Neuron

neural networks, SNN neurons are not tested for activation in every iteration of propagation but only when their membrane potential reaches a certain value, known as threshold. If the membrane level is lower than the threshold, neuron remains off and no output is fired unless the membrane level becomes equal or higher than the threshold. If the membrane level reaches the required level, the neuron is activated and produces a signal that is passed to connected neurons, raising or lowering their membrane potential. There are several SNN models such as the IF (Integrate-and-Fire), Izhikevich, and Hodgkin-Huxley model and LIF (Leaky Integrate-and-Fire) which we target in this paper [10], [20].

A. Field Programmable Gate Arrays (FPGAs)

Field Programmable Gate Arrays (FPGAs) are programmable silicon chips that are commonly designed for specific purposes, known as Domain Specific Architectures (DSAs) [21]. Modern FPGAs consist of millions of logical cells that can be configured to implement a variety of functions. FPGA basic structure consists of look-up tables (LUTs) which perform the logical operations, flip-flops which are registers that store the results of LUTs, interconnections to connect the elements and I/O (IO) blocks to input/output data to and out of the processor elements.

On-chip memories: FPGA architectures are considered System on Chip (SoC) systems where the chip consists of controllers and memories as well as, processor elements, unlike the Von-Neumann architectures (CPUs and GPUs) which are standalone architectures. On-chip memories such as Block RAM (BRAM), Ultra RAM (URAM) and Distributed memories are integrated within the FPGA fabric and thus support low latency accesses. Each Block RAM (BRAM) stores up to 36Kbit of data and can be configured as either two independent 18 Kb RAMs, or one 36 Kb RAM. BRAM and URAM are data memories that reside on chip and don't consume any additional logical resources (LUTs). The bandwidth of BRAM and URAM is above 1000 GB/s, latency is 2 clock cycles

while capacity is 36Mbit [22]. Distributed RAM, on the other hand, is faster than BRAM and URAM, where its latency is 1 clock cycles, however they are built up with LUTs and its capacity is 11 Mbit.

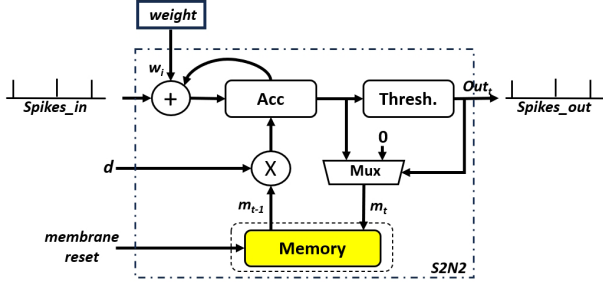


Fig. 2. S2N2 Neuron [10]

IV. ARCHITECTURES

We designed spiking neural network as a two-dimensional array of processor elements (PEs) where each PE represents a S2N2neuron shown in Figure 2. The function of the neuron is illustrated by Equations 1 and 2, where d is the decaying leak parameter, T is the threshold, m_t is the membrane voltage at tick t and w_i is the weight. The yellow box represents the internal storage (memory) that stores the current membrane level. The input to each neuron is represented by spikes that come from the neuron's predecessors. The neurons circuit is designed based on Equation 1 and 2. The accumulator (Acc) and threshold (Thresh) values are stored inside registers (Flip-flops) while the Multiplexer (mux) is implemented using an if-statement. The Multiplication And Addition (MAC) units are programmed as arithmetic function. The memory (yellow block) is where the current membrane level is stored in. The implementation of this unit is designed as a parameter where the user can select if they want to utilize distributed memory, URAM, BRAM or registers based on the hardware size. The network (array) is designed as a systolic array where processors elements are locally connected and perform iterative function synchronously with regular data dependencies. The interconnection between PEs is done as point-to-point connection.

$$m_t = \sum w_i + (1 - out_{t-1}) * d * m_{t-1} \quad (1)$$

$$out_t = 1, if m_t > T, else 0 \quad (2)$$

Figure 3 shows how we implemented the memory allocation for each processor. For example, for 16 neurons (processor elements of 4x4 array), we assign 16 locations of the memory. Each neuron is assigned into a unique address in the memory. The memory index is equivalent to the neuron ID. The memory width is determined by the weight (or data) width while the memory depth is related to the number of neurons, which in turn, will determine the memory capacity. For example, for 256B memory size and 8-bit data width, the memory depth is 32 locations results in 32 neurons (network capacity). However this prediction is theoretical because it does

not take into a consideration the interconnections and the available LUTs, which can limit the network scalability and performance as illustrated in the next section.

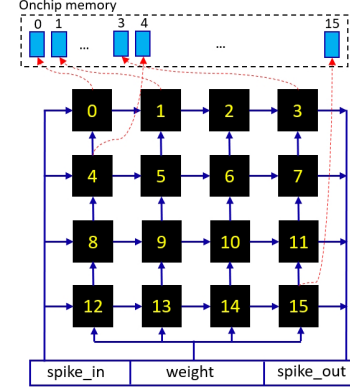


Fig. 3. PE storage assigned into memory location

V. EVALUATION AND RESULTS

SINK is mapped onto commercially available Xilinx Zync104 Ultrascale FPGA. Our proposed SINK is designed using VerilogHDL. The verilog codes are synthesized and implemented in the Xilinx Vivado 2021.2 with ML-based design optimization to achieve a higher clock rate. We evaluated our design on UltraScale Zync104 FPGA device which consists of 225K LUTs, 450K registers, 8MB distributed memory, 312 BRAMs and 96 URAMs [22]. SINK is designed as a customizable accelerator where the network capacity, weight_width and the temporal memory are parameterizable. For testing experiments, we assumed that the number of columns in the network is fixed to 16 columns while the number of rows can vary in the power of 2 range [1-256]. The network capacity can reach up to 256x16 (4096 neurons).

A. Network Evaluation and Trade-offs

The hardware resources that we measured are the Look-up tables (LUTs), registers (Regs.), amount of memory utilized and power consumption. Figures 4 shows the required LUTs and registers utilization to implement four SINK configurations on nine different array size. The LUTs utilization is computed as the percentage of used LUTs over the available LUTs. Similarly, the register utilization is measured. Figures 4 shows that the BRAM and distributed RAM utilize the highest amount of the resources while the URAM consumes the least among the on_chip memories and the registers consumes the least among all.

The estimated power consumption is measured after place-and-route using the power analysis and timing summary tools in the Vivado Design Suite which provides detailed analysis and accurate estimation. Figure 5 shows the consumed power among nine SNN configurations and related to memory type. While in previous figures, BRAM and Distributed memory perform similarly in most of the cases, Figure 5 shows the major difference between the two technologies as distributed

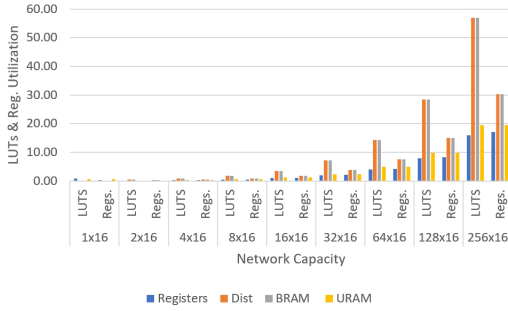


Fig. 4. LUTs and Register Utilization

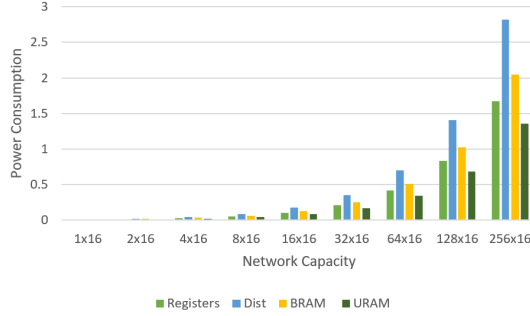


Fig. 5. Power Consumption in Watts

memory consumes higher power comparing to BRAMs. Furthermore, registers consume higher power than URAM, although they utilize less hardware resources as shown in Figures 4. Figure 6 shows the memory utilization among the three SINK on-chip memory configurations.

All of previous results were measured with respect to 8-bit weight_width. SINK performance is proportionally affected by the weight_width. We measured the change in performance on the largest array 256x16. As shown in Figure 7 (which is created on logarithmic scale), the LUTs utilization have significantly increased in register-based and URAM-based SINK while others show a slight increase. Figure 7 also shows the register utilization which increased among all cases.

B. SINK Performance and comparison with State-of-the-arts

The theoretical peak GOP/s of an SNN accelerator is given in Equation 3

$$PeakGOP/s = 2 * f * S \quad (3)$$

where f is the system clock frequency, and $S = M \times N$ denotes the size of the network (systolic array). The peak GOP/s calculation is similar to [5], [17], [23]. In SINK, M and N denotes the number of rows and number of columns of the array respectively. The peak performance should be proportional to the network size which is limited by the number of hardware resources on FPGA device. As shown in Figure 8, the peak GOP is exponentially proportional with the network size and the clock frequency regardless of the memory type.

Table 1 shows the comparison between Firefly [17] and the four SINK configurations. Both SINK and Firefly performs

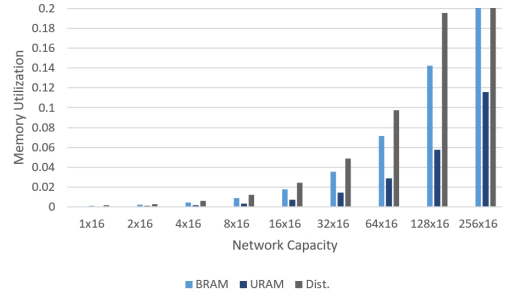


Fig. 6. Memory Utilization.

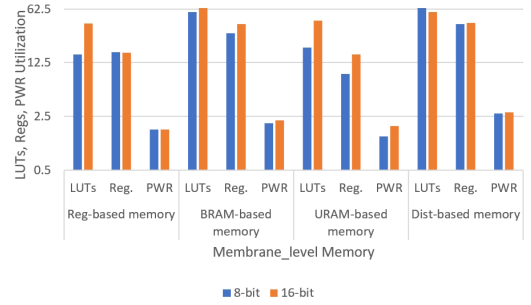


Fig. 7. LUTs, Regs and power Utilization for 8-bit and 16-bit data width

the Output Stationary (OS) of spiking neural network, and use INT8 precision. Firefly [17] utilizes a 144x16 systolic array. To compare with Firefly we ran SINK on an array of 144x16. SINK run on clock frequency of 350MHz while Firefly ran on 300MHz. Firefly achieved 1350 GOP/s on the xczu3eg FPGA device while SINK achieved 1575 GOP/s on zcu104 FPGA devices. While SINK consumes more hardware resources than Firefly in some configurations, Firefly consumes higher on-chip memory. Although some of this memory is allocated for storing the weight, still SINK has considerably low on-chip memory usage specially for the URAM-based configuration.

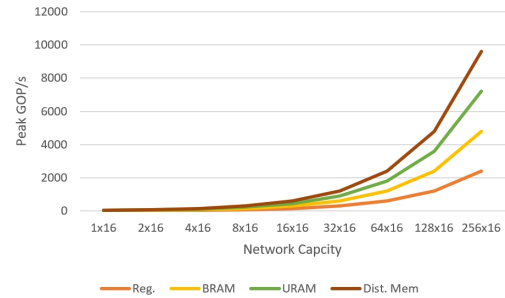


Fig. 8. Peak GOP/s proportionally growing with the array size

VI. CONCLUSION AND FUTURE WORK

In this paper, we introduced a parameterizable Spiking Neural networkK (SINK) accelerator which has the capability to run on different configurations based on the network capacity, weight_width and the membrane_level type of memory. The novelty of this design is the option to implement SNN accelerator using any of the available on-chip memories. We

Table 1 Comparison between SINK and State-of-the-art SNN Accelerators

	FireFly	SINK_Reg	BRAM_SINK	URAM_SINK	Dist_SINK
Device	xczu3eg	zync104	zcu104	zcu104	zcu104
SoC	Ultrascale	Ultrascale	Ultrascale	Ultrascale	Ultrascale
Dataflow	OS	OS	OS	OS	OS
Precision	INT8	INT8	INT8	INT8	INT8
Neuron	IF/LIF	LIF	LIF	LIF	LIF
Array Size	144x16	144x16	144x16	144x16	144x16
Frequency	300MHz	350MHz	350MHz	350MHz	350MHz
Peak GOP/s	1350	1575	1575	1575	1575
LUTs Utilization	21.43%	15%	44.5%	15.3%	32%
Reg. Utilization	—	12%	23.64%	6.95%	17%
B/URAM Utilization	75%	—	0.28%	0.1%	—
Dist. Mem Utilization	—	—	—	—	0.22%

used ZCU104 FPGA device which consists of three on-chip memories, namely distributed memory, Block RAM (BRAM) and UltraRAM(URAM). We performed a tradeoff analysis to measure SINK performance related to each on-chip memory. We found out that URAM can be the best choice in terms of less power consumed, in addition to the lower LUTs and registers used comparing with the distributed memory and BRAM, while the registers consumed less hardware resources among all by highest power consumption than URAM. In terms of scalability, URAM-based SINK can scale up higher given the above results. Comparing with state-of-the-art, SINK could run on higher clock frequency which allows its GOP/s to be higher than Firefly by 1.16x. In terms of the hardware resources, SINK utilizes slightly larger LUTs and registers than Firefly in all configurations. In the future, our goal is to measure SINK accuracy and compare it with the state-of-the-art.

REFERENCES

- [1] A. Carpegna, A. Savino, and S. Di Carlo, "Spiker: an fpga-optimized hardware accelerator for spiking neural networks," in *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2022, pp. 14–19.
- [2] K. Cheung, S. R. Schultz, and W. Luk, "A large-scale spiking neural network accelerator for fpga systems," in *Artificial Neural Networks and Machine Learning–ICANN 2012: 22nd International Conference on Artificial Neural Networks, Lausanne, Switzerland, September 11–14, 2012, Proceedings, Part I* 22. Springer, 2012, pp. 113–120.
- [3] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," *Neural networks*, vol. 121, pp. 294–307, 2020.
- [4] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers in neuroscience*, vol. 12, p. 331, 2018.
- [5] Q. Chen, C. Gao, and Y. Fu, "Cerebron: a reconfigurable architecture for spatiotemporal sparse spiking neural networks," vol. 30, no. 10. IEEE, 2022, pp. 1425–1437.
- [6] I. Fedorov, R. P. Adams, M. Mattina, and P. Whatmough, "Sparse: Sparse architecture search for cnns on resource-constrained microcontrollers," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [7] J. Park and W. Sung, "Fpga based implementation of deep neural networks using on-chip memory only," in *2016 IEEE International conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2016, pp. 1011–1015.
- [8] S. Panchapakesan, Z. Fang, and J. Li, "Syncnn: Evaluating and accelerating spiking neural networks on fpgas," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 4, pp. 1–27, 2022.
- [9] H. Jun, J. Cho, K. Lee, H.-Y. Son, K. Kim, H. Jin, and K. Kim, "Hbm (high bandwidth memory) dram technology and architecture," in *2017 IEEE International Memory Workshop (IMW)*. IEEE, 2017, pp. 1–4.
- [10] A. Khodamoradi, K. Denolf, and R. Kastner, "S2n2: A fpga accelerator for streaming spiking neural networks," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021, pp. 194–205.
- [11] E. Painkras, L. A. Plana, J. Garside, S. Temple, F. Galluppi, C. Patterson, D. R. Lester, A. D. Brown, and S. B. Furber, "Spinnaker: A 1-w 18-core system-on-chip for massively-parallel neural network simulation," *IEEE Journal of Solid-State Circuits*, vol. 48, no. 8, pp. 1943–1953, 2013.
- [12] C.-K. Lin, A. Wild, G. N. Chinya, Y. Cao, M. Davies, D. M. Lavery, and H. Wang, "Programming spiking neural networks on intel's loihi," *Computer*, vol. 51, no. 3, pp. 52–61, 2018.
- [13] R. Karakchi and J. Frierson, "Towards a scalable spiking neural network," in *Proceedings of the Second International Conference on Advances in Computing Research (ACR'24)*, ser. Lecture Notes in Networks and Systems, vol. 956, 2024.
- [14] C. Yakopcic, N. Rahman, T. Atahary, M. Z. Alom, T. M. Taha, A. Beigh, and S. Douglass, "Spiking neural network for asset allocation implemented using the truenorth system," in *2019 IEEE Cognitive Communications for Aerospace Applications Workshop (CCAAS)*. IEEE, 2019, pp. 1–6.
- [15] D. Neil and S.-C. Liu, "Minitaur, an event-driven fpga-based spiking network accelerator," *IEEE transactions on very large scale integration (VLSI) systems*, vol. 22, no. 12, pp. 2621–2628, 2014.
- [16] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "Finn: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*, 2017, pp. 65–74.
- [17] J. Li, G. Shen, D. Zhao, Q. Zhang, and Z. Yi, "Firefly: A high-throughput and reconfigurable hardware accelerator for spiking neural networks," *arXiv preprint arXiv:2301.01905*, 2023.
- [18] J. Smith, "A roadmap for reverse-architecting the brain's neocortex," 2019.
- [19] Z. Li, E. Lemaire, N. Abderrahmane, S. Bilavarn, and B. Miramond, "Efficiency analysis of artificial vs. spiking neural networks on fpgas," *Journal of Systems Architecture*, vol. 133, p. 102765, 2022.
- [20] T. Gao, B. Deng, J. Wang, and G. Yi, "Highly efficient neuromorphic learning system of spiking neural network with multi-compartment leaky integrate-and-fire neurons," *Frontiers in Neuroscience*, vol. 16, p. 929644, 2022.
- [21] M. Gungor, K. Huang, S. Ioannidis, and M. Leeser, "Optimizing designs using several types of memories on modern fpgas," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–7.
- [22] I. Xilinx, "Zynq ultrascale+ mp soc zcu104 evaluation kit," <https://www.xilinx.com/products/boards-and-kits/zcu104.html>, 2019.
- [23] S.-Q. Wang, L. Wang, Y. Deng, Z.-J. Yang, S.-S. Guo, Z.-Y. Kang, Y.-F. Guo, and W.-X. Xu, "Sies: A novel implementation of spiking convolutional neural network inference engine on field-programmable gate array," *Journal of Computer Science and Technology*, vol. 35, pp. 475–489, 2020.